

这是一堂关于搜索算法的 AI 课程，系统讲解了状态空间图与搜索树的区别，并深入剖析了深度优先、广度优先、迭代加深、一致代价、贪心及 A 等搜索算法的工作原理、优缺点及复杂度。重点强调 A 算法需使用**可采纳（乐观）启发函数**才能保证最优解，并举例说明曼哈顿距离等常用启发式。最后提到前沿趋势：用大语言模型自动设计启发函数，甚至用于数学发现。

搜索空间与搜索树的基本概念

状态空间图（State Space Graph）与搜索树（Search Tree）的对比

- 状态空间图中的每个状态（节点）是唯一的，仅出现一次，它完整地刻画了问题的所有可能状态及其转移关系。
- 搜索树中的节点可以多次出现，因为它们对应于从初始状态到该状态的不同路径。同一个状态在搜索树中可能因路径不同而出现在多个位置。
- 搜索树是搜索算法在求解过程中动态构建的“试探性计划”，而状态空间图是对问题本身动态特性的静态描述。
- 在实际应用中，无论是状态空间图还是搜索树，都只会按需构建（as little as possible），而不会一次性构建完整结构。

核心搜索算法框架与策略

通用搜索算法框架

- 所有基于树的搜索算法都可以被归纳为一个通用框架：从初始状态开始，循环地从“边缘”（fringe，即待考虑的候选计划集合）中选择一个节点进行扩展，直到找到目标或确定无解。
- 算法的核心差异仅在于其“探索策略”（exploration strategy），即如何从边缘中选择下一个要扩展的节点。
- 解决方案被定义为从起始状态到目标状态的一系列动作序列。

深度优先搜索（Depth-First Search, DFS）

- **策略**: 优先扩展深度最大的节点。当深度相同时，采用约定（如选择最左侧或字母序最小的节点）。
- **特性分析**:

- **完备性**: 在状态空间无环且最大深度有限的情况下是完备的；若存在无限深的路径，则可能不完备。
- **最优性**: 不保证找到最优解（即步数最少或成本最低的解），因为它只探索树的一个前缀（如最左路径）。
- **时间复杂度**: O(b^m)，其中 b 是分支因子，m 是最大深度。最坏情况下需遍历整棵树。
- **空间复杂度**: O(bm)，只需存储当前路径上的节点及其兄弟节点，空间效率高。

广度优先搜索（Breadth-First Search, BFS）

- **策略**: 优先扩展深度最小的节点，即逐层探索。
- **特性分析**:
  - **完备性**: 如果解存在，则必定能找到（假设分支因子有限）。
  - **最优性**: 在所有行动成本相等（或为常数）的前提下，能保证找到步数最少的最优解。
  - **时间复杂度**: O(b^s)，其中 s 是到最浅解的深度。
  - **空间复杂度**: O(b^s)，需要将当前层及之前所有层的节点都存储在边缘中，空间开销大。

迭代加深搜索（Iterative Deepening Search）

- **策略**: 结合 DFS 和 BFS 的优点，通过反复执行带深度限制的 DFS 来实现。从深度限制 1 开始，若未找到解则递增限制并重试。
- **特性分析**:
  - **完备性与最优性**: 与 BFS 相同，在行动成本为常数时，具备完备性和最优性。
  - **空间复杂度**: 与 DFS 相同，为 O(bd)，其中 d 是当前深度限制，空间效率高。
  - **时间开销**: 虽然会重复访问浅层节点，但由于大部分计算开销集中在最深层，因此总体时间复杂度仍为 O(b^s)，与 BFS 同阶，但空间上更优。

成本敏感搜索算法

一致代价搜索（Uniform Cost Search, UCS）

- **策略**: 优先扩展从起点到该节点的路径总成本（g(n)）最小的节点，而非仅看深度。
- **应用场景**: 适用于行动成本不均等的问题（如罗马尼亚旅行问题中城市间距离不同）。
- **特性分析**:
  - **最优性**: 能保证找到总成本最低的最优解。
  - **探索模式**: 其搜索前沿是一个不规则的多边形，由各节点的实际成本决定，会处理所有成本低于最优解成本的节点。
  - **缺点**: 搜索以起点为中心向四周均匀扩散，不考虑目标方向，可能导致大量无效探索。

启发式搜索与 A*算法

- **启发式函数（Heuristic Function, h(n)）**:
  - 定义为从当前状态 n 到目标状态的估计成本。它是问题特定的，需要人工设计（如曼哈顿距离、欧氏距离）。
  - **可采纳性（Admissibility）**: 一个关键性质，要求 h(n)永远不大于从 n 到目标的实际成本。乐观的（非高估的）启发式函数是可采纳的。
- **贪婪最佳优先搜索（Greedy Best-First Search）**:
  - **策略**: 仅根据启发式函数 h(n)选择下一个节点，即选择看起来离目标最近的节点。
  - **优缺点**: 效率高但不完备也不保证最优，性能极度依赖启发式函数的准确性。
- **A*搜索算法**:
  - **策略**: 结合了 UCS 的保守性和贪婪搜索的方向性，通过评估函数 f(n) = g(n) + h(n)选择节点，其中 g(n)是已发生的实际成本，h(n)是未来的估计成本。

- **最优性保证**: 当启发式函数 h(n)是可采纳的（即乐观的）时，A*算法既能保证完备性，也能保证找到最优解。
- **应用**: A*是实践中最常用的搜索算法之一，广泛应用于路径规划等领域。

现代 AI 与启发式函数的自动设计

AI 驱动的算法设计新范式

- **背景**: 传统上，启发式函数需要领域专家针对每个问题手动设计，这是一个耗时且需要深厚专业知识的过程。
- **新方法**: 研究人员（如 Google DeepMind）已开始利用大型语言模型（LLM）将启发式函数的设计本身形式化为一个搜索问题。
- **工作流程**:
  - 将问题描述和现有优秀启发式函数作为输入提供给 LLM。
  - LLM 生成新的启发式函数变体（通常以代码形式）。
  - 在模拟环境中评估新函数的性能（如在在线装箱问题中使用的箱子数量）。
  - 根据评估结果，将表现好的函数保留并加入数据库，用于下一轮迭代优化。
- **成果**: 此方法已在在线装箱等问题上发现优于人类设计的启发式策略，并成功应用于数学发现等领域，标志着算法设计正进入人机协同甚至自动化的新阶段。

待办事项

- @学生 阅读并运行本周提供的 Python 教程代码，亲自验证各类搜索算法（DFS, BFS, UCS, A*等）在“罗马尼亚旅行”问题上的行为和性能。
- @学生 思考并验证“煎饼排序”问题中“错位的最大煎饼索引”这一启发式函数是否满足可采纳性。

会议分析

- **核心主旨**: 本次讲座系统性地讲解了人工智能中经典搜索算法的核心概念、工作原理、性能特性（完备性、最优性、时空复杂度）以及从盲目搜索到启发式搜索的演进逻辑，并展望了利用 AI 自动设计算法（特别是启发式函数）的前沿趋势。
- **逻辑分析**: 讲座首先通过区分状态空间图和搜索树建立基础认知，然后引入通用搜索框架，指出算法差异仅在于节点选择策略。接着，通过分析 DFS 和 BFS 的优缺点，自然引出迭代加深搜索作为折中方案。随后，为解决现实世界中成本不均的问题，引入 UCS，并因其缺乏方向性而进一步引出启发式搜索，最终聚焦于兼具效率与最优性保证的 A*算法，并强调了可采纳启发式的关键作用。
- **关键重点**: 1. 状态空间图（唯一状态）与搜索树（路径相关）的根本区别；2. A*算法的最优性严格依赖于启发式函数的可采纳性（即不能高估）；3. 利用大型语言模型将算法设计（如启发式函数）本身转化为一个可被自动搜索和优化的问题，是当前 AI 研究的重要前沿。
- **潜台词与趋势**: 讲座暗示，未来 AI 工程师的核心能力可能不仅是应用现有算法，更是能够定义和引导 AI 系统去自动发现和优化解决问题的新算法。这种“AI for AI”的元学习范式，正在成为推动计算机科学乃至数学等领域突破的新动力。